

ООП 1

ОБЕКТНО ОРИЕНТИРАН ДИЗАЙН 1

ЗАЩО ИЗУЧАВАМЕ ОБЕКТЕН ДИЗАЙН?

- ЗА ДА ПРЕВЪРНЕМ РЕАЛЕН ПРОБЛЕМ В КОД
- ЗА ДА ИЗБЕГНЕМ ЛОШИ СТРУКТУРИ „ВСИЧКО В ЕДИН КЛАС“
- ЗА ДА ПРАВИМ СИСТЕМИ, КОИТО МОГАТ ДА СЕ РАЗШИРЯВАТ
- ЗА ДА РАБОТИМ С ДАННИ КАТО С РЕАЛНИ ОБЕКТИ (ДОМЕЙН МОДЕЛ)
- ЗА ДА ИЗБЕРЕМ ПРАВИЛНАТА КОЛЕКЦИЯ: LIST? SET? MAP?

КАЗУС: БИБЛИОТЕКА, КОЯТО СЪХРАНЯВА КНИГИ И ЧИТАТЕЛИ. ПРИМЕР ЗА ЛОШ ДИЗАЙН

```
PUBLIC CLASS LIBRARYSYSTEM {  
  
    STRING[] TITLES = NEW STRING[100];  
    STRING[] AUTHORS = NEW STRING[100];  
    BOOLEAN[] AVAILABLE = NEW BOOLEAN[100];  
  
    INT BOOKCOUNT = 0;  
  
    VOID ADDBOOK(STRING TITLE, STRING AUTHOR) {  
        TITLES[BOOKCOUNT] = TITLE;  
        AUTHORS[BOOKCOUNT] = AUTHOR;  
        AVAILABLE[BOOKCOUNT] = TRUE;  
        BOOKCOUNT++;  
    }  
}
```

ПРОБЛЕМИ

- ДАННИТЕ ЗА КНИГАТА СА РАЗКЪСАНИ В 3 МАСИВА
- МАСИВИТЕ ТРЯБВА ДА СА *ВИНАГИ* СИНХРОНИЗИРАНИ ПО ИНДЕКС
- НУЖЕН Е РЪЧЕН БРОЯЧ
- ВСЕКИ ПРОПУСК (ПРИМЕРНО BOOKCOUNT++) → ТОТАЛНА ГРЕШКА
- НЯМА КЛАС BOOK → НЯМА ИНКАПСУЛАЦИЯ
- РАЗШИРЯВАНЕТО Е КОШМАР (ЖАНР? ISBN? ДАТА?)
- ДИЗАЙНЪТ НЕ СЪОТВЕТСТВА НА РЕАЛНИЯ ДОМЕЙН

ПРЕМИНАВАНЕ КЪМ ОБЕКТНО МОДЕЛИРАНЕ

- СЪПКА 1: ИЗВЛИЧАМЕ СЪЩЕСТВИТЕЛНИТЕ → КЛАСОВЕ:

- **BOOK**
- **READER**
- **LIBRARY**

- СЪПКА 2: СВЪРЗВАМЕ ГИ ЧРЕЗ ОТНОШЕНИЯ:

- LIBRARY ИМА BOOK → КОМПОЗИЦИЯ?
- LIBRARY ИМА READER → КОМПОЗИЦИЯ/АГРЕГАЦИЯ
- BOOK НЕ ПОЗНАВА LIBRARY
- READER НЕ ПОЗНАВА LIBRARY

- СЪПКА 3: ДЕФИНИРАМЕ ПОВЕДЕНИЕ:

- `ADDBOOK()`, `REGISTERREADER()`, `LENDBOOK()`

ДОБЪР ДИЗАЙН (СЛЕД): КЛАС BOOK

```
PUBLIC CLASS BOOK {  
  
    PRIVATE FINAL STRING INVENTORYNUMBER;  
  
    PRIVATE FINAL STRING TITLE;  
  
    PRIVATE FINAL STRING AUTHOR;  
  
    PRIVATE BOOLEAN AVAILABLE = TRUE;  
  
    PUBLIC BOOK(STRING INVENTORYNUMBER, STRING TITLE, STRING AUTHOR) {  
  
        THIS.INVENTORYNUMBER = INVENTORYNUMBER;  
  
        THIS.TITLE = TITLE;  
  
        THIS.AUTHOR = AUTHOR;  
  
    }  
  
    PUBLIC BOOLEAN ISAVAILABLE() { RETURN AVAILABLE; }  
  
    PUBLIC VOID MARKUNAVAILABLE() { AVAILABLE = FALSE; }  
  
    PUBLIC VOID MARKAVAILABLE() { AVAILABLE = TRUE; }  
  
    PUBLIC STRING GETINVENTORYNUMBER() { RETURN INVENTORYNUMBER; }  
  
    @OVERRIDE  
    PUBLIC BOOLEAN EQUALS(OBJECT O) {  
  
        IF (THIS == O) RETURN TRUE;  
  
        IF (!(O instanceof BOOK)) RETURN FALSE;  
  
        RETURN INVENTORYNUMBER.EQUALS(((BOOK) O).INVENTORYNUMBER);  
  
    }  
  
    @OVERRIDE  
    PUBLIC INT HASHCODE() {  
  
        RETURN INVENTORYNUMBER.HASHCODE();  
  
    }  
  
}
```

ДОБЪР ДИЗАЙН: КЛАС READER

```
PUBLIC CLASS READER {  
    PRIVATE FINAL STRING NAME;  
    PRIVATE FINAL STRING CARDNUMBER;  
  
    PUBLIC READER(STRING NAME, STRING CARDNUMBER) {  
        THIS.NAME = NAME;  
        THIS.CARDNUMBER = CARDNUMBER;  
    }  
  
    PUBLIC STRING GETCARDNUMBER() { RETURN CARDNUMBER; }  
}
```

ДОБЪР ДИЗАЙН: КЛАС LIBRARY (ОСНОВЕН МОДЕЛ)

```
PUBLIC CLASS LIBRARY {
```

```
    PRIVATE FINAL MAP<STRING, BOOK> CATALOG = NEW HASHMAP<>();
```

```
    PRIVATE FINAL MAP<STRING, READER> READERS = NEW HASHMAP<>();
```

```
    PUBLIC VOID ADDBOOK(BOOK BOOK) {
```

```
A        CATALOG.PUT(BOOK.GETINVENTORYNUMBER(), BOOK);
```

```
    }
```

```
    PUBLIC VOID REGISTERREADER(READER READER) {
```

```
        READERS.PUT(READER.GETCARDNUMBER(), READER);
```

```
    }
```

```
    PUBLIC VOID LENDBOOK(STRING INVENTORYNUMBER, STRING CARDNUMBER) {
```

```
        BOOK BOOK = CATALOG.GET(INVENTORYNUMBER);
```

```
        READER READER = READERS.GET(CARDNUMBER);
```

```
        IF (BOOK == NULL || READER == NULL)
```

```
            THROW NEW ILLEGALARGUMENTEXCEPTION("NOT FOUND");
```

```
        IF (!BOOK.ISAVAILABLE())
```

```
            THROW NEW ILLEGALSTATEEXCEPTION("UNAVAILABLE");
```

```
        BOOK.MARKUNAVAILABLE();
```

```
    }
```

```
    PUBLIC VOID RETURNBOOK(STRING INVENTORYNUMBER) {
```

```
        BOOK BOOK = CATALOG.GET(INVENTORYNUMBER);
```

```
        IF (BOOK == NULL) THROW NEW ILLEGALARGUMENTEXCEPTION("BOOK NOT FOUND");
```

```
        BOOK.MARKAVAILABLE();
```

```
    }
```

```
}
```

ПРОЦЕДУРНО VS ОБЕКТНО

Елемент	❌ ПРЕДИ	✅ СЛЕД
Представяне	масиви	обекти
Уникалност	няма	inventoryNumber
Търсене	линейно	$O(1)$
Състояние	отделни масиви	вътре в Book
Разширяемост	лоша	отлична
Инкапсулация	липсва	налична
Модел	процедурен	обектен

КОЙ ТИП КОЛЕКЦИЯ ДА ИЗПОЛЗВАМЕ? (LIST, SET, MAP)

1) LIST

- ПОЗВОЛЯВА ДУБЛИКАТИ
- ЗАПАЗВА РЕД
- ПОДХОДЯЩО ПРИ МНОГО КОПИЯ НА ЕДНА И СЪЩА КНИГА
- (ПРИМЕРНО 3 ЕКЗЕМПЛЯРА НА „RETURN OF THE JEDI“)

КОЙ ТИП КОЛЕКЦИЯ ДА ИЗПОЛЗВАМЕ? (LIST, SET, MAP)

2) SET

- НЯМА ДУБЛИКАТИ
- МНОГО БЪРЗО ТЪРСЕНЕ (HASHSET)
- ПОДХОДЯЩО, АКО УНИКАЛНОСТТА Е В EQUALS/HASHCODE (ПРИМЕРНО INVENTORYNUMBER)

КОЙ ТИП КОЛЕКЦИЯ ДА ИЗПОЛЗВАМЕ? (LIST, SET, MAP)

3) MAP<STRING, BOOK>

- КЛЮЧ → BOOK
- НАЙ-ЯСЕН МОДЕЛ ЗА КАТАЛОЗИ
- LOOKUP Е $O(1)$
- НЕ ЗАВИСИ ОТ EQUALS/HASHCODE

КОГА КОЙ ТИП СЕ ПОЛЗВА?

Ситуация	Най-добър избор
Колекция от уникални книги	Set
Каталог по номера	Map<String, Book>
Много копия на една книга	List
Изисква се бързо търсене по ключ	Map
Знаем equals/hashCode са дефинирани добре	Set

ПРИМЕР: SET

```
SET<BOOK> BOOKS = NEW HASHSET<>();  
BOOKS.ADD(NEW BOOK("BG001", "JAVA", "IVANOV"));  
BOOKS.ADD(NEW BOOK("BG001", "JAVA", "IVANOV")); // няма да се добави втори път
```

ПРИМЕР: LIST (МНОГО КОПИЯ)

```
LIST<BOOK> BOOKS = NEW ARRAYLIST<>();  
BOOKS.ADD(NEW BOOK("BG001", "JAVA", "IVANOV"));  
BOOKS.ADD(NEW BOOK("BG002", "JAVA", "IVANOV")); // ДВЕ РАЗЛИЧНИ КОПИЯ
```

ПРИМЕР: MAP<STRING, BOOK> (ΚΑΤΑΛΟΓ)

```
MAP<STRING, BOOK> CATALOG = NEW HASHMAP<>();  
CATALOG.PUT("BG001", NEW BOOK("BG001", "JAVA", "IVANOV"));  
  
BOOK BOOK = CATALOG.GET("BG001");
```

COUPLING & COHESION

- ВИСОКА КОХЕЗИЯ → КЛАСЪТ ИМА ЯСНО ПРЕДНАЗНАЧЕНИЕ
- НИСЪК COUPLING → КЛАСОВЕТЕ ЗНАЯТ МАЛКО ЕДИН ЗА ДРУГ
- ПРИМЕР ЗА ЛОШ COUPLING:
 - `LIBRARY.CATALOG.GET("BG001").AVAILABLE = FALSE;`
- ПРИМЕР ЗА ДОБЪР:
 - `LIBRARY.LENDBOOK("BG001", "R123");`

ЛОШИЯТ ПРИМЕР: КЪДЕ Е ПРОБЛЕМЪТ?

- `LIBRARY.CATALOG.GET("BG001").AVAILABLE = FALSE;`
1. ДОСТЪПВАМЕ ДИРЕКТНО ВЪТРЕШНАТА СТРУКТУРА `CATALOG`, КОЕТО НАРУШАВА ИНКАПСУЛАЦИЯТА.
 2. ДОСТЪПВАМЕ ОБЕКТ ПО КЛЮЧ, КОЕТО НИ ВРЪЩА `BOOK`.
 3. ПРОМЕНЯМЕ ДИРЕКТНО ЧАСТНО ПОЛЕ (`AVAILABLE`), КОЕТО ПО ПРИНЦИП ТРЯБВА ДА СЕ УПРАВЛЯВА САМО ОТ МЕТОДИ.
 4. ПРЕСКАЧАМЕ ЛОГИКАТА, КОЯТО `LIBRARY` ТРЯБВА ДА КОНТРОЛИРА (ПРОВЕРКА ЗА ЧИТАТЕЛ, СТАТУС, ПРАВИЛА И ДР).

ЛОШИЯТ ПРИМЕР: КЪДЕ Е ПРОБЛЕМЪТ?

- ТОВА Е СИЛНА СВЪРЗАНОСТ (STRONG COUPLING) И НИСКА КОХЕЗИЯ, ЗАЩОТО:
 - ВЪНШНИЯТ КОД ЗНАЕ КАК LIBRARY ПАЗИ КНИГИ
 - ВЪНШНИЯТ КОД ЗНАЕ СТРУКТУРАТА НА BOOK
 - ВЪНШНИЯТ КОД ПРОМЕНЯ ВЪТРЕШНО СЪСТОЯНИЕ, КОЕТО LIBRARY НЕ КОНТРОЛИРА
 - АКО ПРОМЕНИМ ИМПЛЕМЕНТАЦИЯТА ВЪТРЕ В LIBRARY → ТУК ЩЕ СЕ СЧУПИ

НАРУШЕНИЕ НА ИНКАПСУЛАЦИЯТА

- AVAILABLE Е ЧАСТНО ПОЛЕ
 - ТО ТРЯБВА ДА СЕ УПРАВЛЯВА ОТ МЕТОДА:
 - `BOOK.MARKUNAVAILABLE()`;
 - НО ЛОШИЯТ КОД ГО ЗАОБИКАЛЯ
 - НАРУШАВАМЕ ОСНОВНОТО ПРАВИЛО НА ООП:
 - **СЪСТОЯНИЕТО НА ОБЕКТА СЕ ПРОМЕНЯ САМО ЧРЕЗ НЕГОВИТЕ СОБСТВЕНИ МЕТОДИ.**

LIBRARY РАЗКРИВА ВЪТРЕШНОТО СИ ПРЕДСТАВЯНЕ (INTERNAL REPRESENTATION LEAK)

- КОГАТО ПИШЕМ:
 - `LIBRARY.CATALOG.GET("BG001")`
- ТОВА ОЗНАЧАВА, ЧЕ ВЪНШНИЯТ КОД:
 - ЗНАЕ, ЧЕ LIBRARY ИЗПОЛЗВА MAP
 - ЗНАЕ КАК СЕ КАЗВА (CATALOG)
 - ЗНАЕ КЛЮЧА
 - ЗНАЕ, ЧЕ BOOK ВЪТРЕ ИМА ПОЛЕ AVAILABLE
- LIBRARY ВЕЧЕ НЕ МОЖЕ ДА ПРОМЕНИ ВЪТРЕШНАТА СИ РЕАЛИЗАЦИЯ, ЗАЩОТО ВСИЧКИ ПОТРЕБИТЕЛИ ЗАВИСЯТ ОТ НЕЯ.

СЛЕДСТВИЕ

- АКО СЕ РЕШИ:
 - ДА СЕ СМЕНИ MAP С SET
 - ИЛИ ДА СЕ ПАЗИ BOOK В БАЗА
 - ИЛИ ДА СЕ ФИЛТРИРА НЕДОСТЪПНИ КНИГИ
- → ЦЯЛАТА СИСТЕМА СЕ ЧУПИ, ЗАЩОТО МНОГО ВЪНШНИ КЛАСОВЕ БЪРКАТ ДИРЕКТНО В CATALOG.
- ТОВА Е НАГЛЕДЕН ПРИМЕР ЗА STRONG COUPLING.

ПРЕСКАЧАНЕ НА ВАЛИДИРАЩАТА ЛОГИКА

- `PUBLIC VOID LENDBOOK(String inventoryNumber, String cardNumber)`
- ПРОВЕРЯВА ДАЛИ ЧИТАТЕЛ СЪЩЕСТВУВА
- ПРОВЕРЯВА ДАЛИ КНИГАТА Е НАЛИЧНА
- ПРОВЕРЯВА ДАЛИ ИНВЕНТАРНИЯТ НОМЕР Е ВАЛИДЕН
- ПРИ НУЖДА ХВЪРЛЯ ИЗКЛЮЧЕНИЯ

ПРЕСКАЧАНЕ НА ВАЛИДИРАЩАТА ЛОГИКА

- Но лошият код:
 - ПРЕСКАЧА ВСЯКА ЛОГИКА И ПРОСТО БЪРКА В ПОЛЕТО.
- РЕЗУЛТАТ:
 - МОЖЕШ ДА МАРКИРАШ КНИГА КАТО UNAVAILABLE БЕЗ ЧИТАТЕЛ
 - МОЖЕШ ДА НАПРАВИШ НЕВАЛИДНИ СЪСТОЯНИЯ
 - МОЖЕШ ДА НАПРАВИШ КНИГА НЕДОСТЪПНА БЕЗ ПРОВЕРКА

НЕКОНСИСТЕНТНО СЪСТОЯНИЕ НА ОБЕКТ

- `BOOK.AVAILABLE = FALSE;`
- LIBRARY МОЖЕ ДА ИМА КНИГА В “ЗАЕМАНЕ”, НО БЕЗ ЧИТАТЕЛ
- МОЖЕ ДА СЕ НАПРАВИ ДЕПОЗИРАНА КНИГА НЕДОСТЪПНА
- МОЖЕ ДА СЕ НАРУШАТ ИНВАРИАНТИ
- МОЖЕ ДА СЪЗДАДЕШ ЛОГИЧЕСКИ НЕВЪЗМОЖНИ СЪСТОЯНИЯ
- НАЙ-ГОЛЕМИЯТ ПРОБЛЕМ НА СИЛНАТА СВЪРЗАНОСТ:
- ВЪНШНИЯТ КОД МОЖЕ ДА НАПРАВИ LIBRARY ЛОГИЧЕСКИ НЕВАЛИДНА.

ЗАЩО ВТОРИЯТ ВАРИАНТ Е ДОБЪР?

- `LIBRARY.LENDBook("BG001", "R123");`
- LIBRARY РЕШАВА КАКВО СТАВА
 - LIBRARY ВАЛИДИРА ВХОДА
 - LIBRARY ПРОМЕНЯ STATE САМО ПРЕЗ МЕТОДИ
 - BOOK ПРОМЕНЯ STATE САМО ПРЕЗ `MARKUNAVAILABLE()`
 - НЯМА ДИРЕКТЕН ДОСТЪП ДО ВЪТРЕШНИ СТРУКТУРИ
 - НЯМА “LEAK” НА ИМПЛЕМЕНТАЦИЯ
 - НЯМА НАРУШЕНИЕ НА ИНВАРИАНТИ

ОБОБЩЕНИЕ

- МАСИВИ → ЛОШ МОДЕЛ
- ОБЕКТНО МОДЕЛИРАНЕ → ЯСНА СТРУКТУРА
- LIST, SET, MAP → РАЗЛИЧНИ СЕМАНТИКИ
- MAP Е ЕСТЕСТВЕН ЗА КАТАЛОЗИ
- SET Е ИДЕАЛЕН ПРИ УНИКАЛНОСТ (EQUALS/HASHCODE)
- LIST Е ИДЕАЛЕН ПРИ КОПИЯ
- ДОБЪР ДИЗАЙН ПРАВИ КОДА ПО-ЛЕСЕН ЗА ПОДДРЪЖКА